

Generalised Tracking-by-Detection Multiple-Object Tracking Framework

October 10, 2025

1 Introduction

In recent years, the field of single-camera Multiple Object Tracking (MOT) has seen significant advances, particularly in the development of real-time or ‘online’ MOT systems. Online tracking approaches process incoming frames sequentially (frame-by-frame) without access to future information, in contrast to offline tracking approaches that have access to past and future frames [LXM⁺21]. In this paper, we focus on single camera applications, as multi-camera applications have additional challenges.

SORT (Simple Online and Realtime Tracking) [BGO⁺16] is a seminal work on online MOT. This approach combines an *Object Detector* to locate object-bounding boxes, a Kalman Filter to predict future positions of tracked objects [LRJD15], and the Hungarian Algorithm [Kuh55] to solve the linear assignment problem. SORT inspired numerous tracking algorithms that employ an object detector to localise objects and extract object information, and tracking these objects using linear assignment. This type of MOT systems are known as **Tracking-by-Detection** (TbD) approaches, and are specifically suited for online MOT.

Some notable online TbD MOT systems inspired by SORT include *DeepSORT* (2017), which extended SORT by incorporating deep learning-based ReID (Re-Identification) features to improve tracking given occlusions and reappearances [WBP17]. *BYTETrack* (2021), which enhanced tracking performance by associating low-confidence detections, allowing the system to recover missed objects and maintain more complete object trajectories [ZSJ⁺22]. Building on these techniques, *StrongSORT* (2022) and *SMILETrack* (2022) introduced further innovations to handle occlusions, visually similar objects, and missed associations with greater accuracy and speed [DZS⁺23, WHC⁺24]. Shifting focus toward improving the prediction model, *OC-SORT* (2022) addressed non-linear object motion and occlusions using observation-centric methods [CPW⁺23], while *BoT-SORT* (2022) combined motion and appearance information with camera-motion compensation and introduced a more accurate Kalman filter state vector [AOB22]. When released, all of these methods achieved competitive

results on public MOT benchmarks - MOT16, MOT17, MOT20, DanceTrack and KITTI [DOM+20, DRM+20, SCJ+22, LXG22].

Pose Tracking (PT), a task which involves detecting human poses followed by frame-to-frame association, is often regarded as distinct from MOT being more closely associated with Human Pose Estimation (HPE), or Action Recognition [ZML+23]. Zhou et al. [ZML+23] proposed applying ByteTrack, an online TbD MOT system, to the task of Pose Tracking (PT) as a direction for future work [ZSJ+22]. Some current works in PT draw inspiration from online TbD MOT systems, notably *AlphaPose* and *LightTrack* [FLT+22, NH19]. Both systems perform HPE to detect human poses followed by a tracking procedure, which is comparable to how TbD MOT systems perform Object Detection followed by a tracking procedure. This motivated us to explore how both MOT and PT can be viewed as similar tasks.

Given the structural similarities between the TbD MOT, PT, and TbD MOT systems themselves, such as those based on SORT; we believe that a generalised framework can be established. Both MOT and PT rely on detection followed by frame-to-frame association, utilizing similar tools (e.g., Kalman Filters and assignment algorithms). A generalised framework could simplify development and provide a flexible and scalable solution for both MOT and PT tasks.

In this paper, we aim to reframe the task of TbD MOT for Computer Vision within a generalised context. The contributions of this paper can be summarised as follows:

- Formalization of the tracking task as identity-propagation across time, addressing the similarities between MOT and PT by leveraging detection and frame-to-frame association, techniques shared by both tasks.
- Definition of a generic framework that describes the core components of a TbD system, including the temporal component, the detection component, and the tracking component based on the formalization and the ideas presented in TbD MOT systems and PT systems [BGO+16, WBP17, DZS+23, ZSJ+22, WHC+24, AOB22, CPW+23, FLT+22, NH19], we

In Section 2, we present the general idea of a TbD MOT system and the provide intuition on how to decompose a TbD MOT into a *Temporal*, *Detection*, and *Tracking* Component. In Sections ??, ??, and 5 we explain the Temporal Component, Detection Component and Tracking Component respectively.

2 Designing Tracking-by-Detection

A TbD MOT approach is defined as a computational framework designed to detect multiple objects of prespecified classes (such as people or cars) within a sequence of frames in a video, and then tracking these objects by assigning and propagating a unique ID across frames [LXM+21].

Since TbD MOT systems build upon the detections obtained from a detector, the performance of these systems is directly influenced by the quality of the

object detector [LXM⁺21]. Therefore, to fully understand TbD MOT, it is crucial to first understand object detection and how its outputs serve as inputs for tracking.

2.1 Object Detection

Object Detection is the task of localising objects within an image and assigning them class labels [DHZL22]. Given an input image frame, the object detector outputs a set of *bounding boxes*, *class labels*, and *confidence scores* for the detected objects.

A bounding box defines the spatial position of the detected object within the frame. Each detected object is also assigned a class label indicating its type, such as a person, car, or bicycle. For instance, humans are often assigned a label of 0. Along with the bounding box and label, the detector provides a confidence score between 0 and 1, representing the likelihood that the detection is correct, with higher values indicating greater confidence.

2.2 Tracking

While object detection localises and classifies objects in individual frames, it is not able to associate objects across different timesteps i.e. in consecutive frames. Object detection is extended to a TbD MOT system by incorporating temporal context, which allows systems to track objects across frame by propagating their identity over time [LXM⁺21]. We design a TbD MOT system around these three fundamental ideas - *Time* (Temporal component), *Identity* (Detection component) and *Propagation* (Tracking component), where each idea will contribute to a component of a generalised TbD MOT system.

The components of a generalised TbD MOT system are defined as the Temporal Component, Detection Component, and Tracking Component.

- **Temporal Component** - MOT is inherently a temporal problem, where objects must be tracked across a sequence of timesteps (frames). The temporal component is responsible for processing the frames in a video while maintaining temporal continuity.
- **Detection Component** - The detection component is used to localise the objects in the current frame, and extract properties from each object. The properties extracted are used to calculate some similarity score that determine which objects are similar, in order to establish identity.
- **Tracking Component** - The tracking component associates currently detected objects with previously detected objects based on the properties extracted by the detection component. Based on this association, identity is propagated to the current timestep. Track management techniques are also handled by the tracking component to account for any potential errors during the tracking process.

3 Temporal Component

The temporal component of the system is implemented as a video processing module that takes a video sequence as input and outputs its frames in consecutive order. This component manages timesteps, with each timestep corresponding to the processing of a frame. However, it is important to note that timesteps and frames are distinct concepts. While each timestep typically processes the next frame in sequence, some architectures, such as LightTrack [NH19], may skip frames to improve computational efficiency, processing every n^{th} frame instead. This means that a timestep is not always tied to the frame number, and a system can be designed to skip frames and only process a subset of them.

When skipping frames the overall similarity between objects in consecutive timesteps tends to decrease, as objects may move further apart, change appearance or leave/enter the scene, making it harder to establish identity. Thus, skipping leads to a trade-off between accuracy and efficiency, and this depends on a number of factors such as the available resources to run the system, or the frame rate of the video sequence. Ideally, frames processed at consecutive timesteps should be **temporally correlated**, which means that the frames should be sufficiently close enough in time such that the identity information is preserved. This implies that skipping should be a last resort, and that only a small number of frames should be skipped between timesteps. Objects detected in temporally correlated frames will be referred to as temporally correlated objects.

There are scenarios where the identity information between temporally correlated frames is not preserved. These scenarios exist due to the positioning of the camera used, a single-camera will always be limited by the frame size used and its perspective. We will call these potential issues as the **Camera Effects**.

- An *object may enter* the scene in the current frame, introducing a new identity.
- An object may *leave the scene* in the current frame, resulting in a lost identity.
- An object may be occluded.
- An object may *re-enter* the scene after leaving, allowing the system to remember and recover its identity.

Assumption 1 *We assume that temporally correlated objects corresponding to the same identity experience minimal temporal change. Conversely, this implies that **temporally correlated objects, which are sufficiently similar, correspond to the same identity**.*

The problem of TbD MOT can be boiled down to associating temporally correlated objects based on their similarity in order to establish identity.

4 Detection Component

The detection component is responsible for identifying objects in each frame, and measuring their properties. The measured properties are then used to compute a similarity metric, which is able to quantify the degree of similarity between temporally correlated objects. In this section we will explore properties in more detail, how they are measured, the different similarity metrics based on these properties, and how dissimilarity can be derived from similarity.

4.1 Properties

An object can be described as a **set of measurable properties**. By Assumption 1, temporally correlated objects that correspond to the same identity should experience minimal change. Therefore, we can extend the assumption so that properties which are indicative of object identity should also exhibit minimal change. We can categorise properties into two types: **Identity Preserving Properties (IPPs)**, and **Non-Identity Preserving Properties (NIPPs)**.

IPPs are measured properties/features that are expected to exhibit minimal change for a given object identity across temporally correlated frames. These properties are indicators of similarity between objects, and are associated with a corresponding similarity metric which will be discussed further in Section ??.

Some IPPs to note are the following.

- **Position:** The spatial location of the object in the frame, typically represented by a bounding box that encloses the object.

typically represented in one of several formats. Note that in computer vision tasks, the y-axis is reversed so the top-left corner of the image is $(0, 0)$.

- (x_1, y_1, w, h) , where (x_1, y_1) is the top-left corner and (w, h) are the width and height of the box.
- (cx, cy, w, h) , where (cx, cy) is the centre of the bounding and (w, h) are the width and height of the box.
- (x_1, y_1, x_2, y_2) , where (x_1, y_1) is the top-left coordinate and (x_2, y_2) is the bottom-right coordinate.

- **Appearance:** It is common to use a *Re-ID network* to extract appearance features, which outputs a high-dimensional vector corresponding to the appearance of an object.

Different methods handle appearance features differently. *DeepSORT* computes the cosine distance between appearance features extracted from a pre-trained CNN [WBP17]. *SMILEtrack* uses a Similarity Learning Module with a Siamese network and Patch Self-Attention blocks, leveraging Transformer-inspired techniques [WHC⁺24]. *AlphaPose* [FLT⁺22] integrates a re-id system with a pose estimator to create a pose-guided attention to focus more on extracting features related to the human.

- **Keypoints:** Distinct landmarks such as facial features or joint positions in human pose estimation. These are usually represented as a list of keypoints, where each keypoint corresponds to a specific landmark.

A keypoint $k \in K$ contains the x, y coordinates of the point along with a value c which determines the confidence score of the keypoint. Pose information of an individual is represented as a set of keypoints, each keypoint corresponding to a specific joint, so the pose $P = k_1, \dots, k_{17}$, with 17 being the total number of keypoints in the system. Note that each pose will also have a corresponding bounding box which encloses it.

NIPPs are properties that provide additional heuristic-based information to assist in filtering and matching objects during the tracking procedure. While they do not directly indicate identity, they are used to improve the accuracy of tracking. These properties include.

- **Confidence Score:** A measure of the detector’s certainty about the object detection. Used by ByteTrack in the tracking procedure [ZSJ⁺22].
- **Track Age:** The number of times a track has consistently matched, indicating a greater likelihood of matching. Used by DeepSORT in the tracking procedure [WBP17].

4.2 Detectors

In the context of a generalised tracking system, a **detector** is any tool that outputs object information from a given frame, with key examples being machine learning and deep learning tools. Table 1 provides an overview of commonly used detectors.

Detector	Localisation	IPP	NIPP
Object Detector	Yes	Bounding Box	Object Class, Confidence Score
ReID Network	No	Appearance Embedding	N/A
Human Pose Estimator	Yes	Keypoints	Keypoint Confidence Score

Table 1: A list of commonly used detectors in tracking systems, describing whether they are able to localise objects, the IPPs measured, and the NIPPs measured.

Different detectors measure different object properties, and combining these properties in meaningful ways may result in better tracking accuracy. Algorithms like *DeepSORT*, *SmileTrack*, and *StrongSORT* combined position and appearance information [WBP17, WHC⁺24, DZS⁺23], while *AlphaPose* combined position, appearance and keypoint information [FLT⁺22].

The performance of a MOT system is heavily dependent on the accuracy of the detectors used [LXM⁺21]. Errors made by the detector propagate into the tracking component and may affect the overall tracking accuracy. The errors experienced by a detector, which we will call the **Detector Effects**, are

Incorrect Detections (False Positives) where Non-object entities are mistakenly detected as objects, and *Missed Detections* (False Negatives) where objects that exist in the frame are not detected by the system. Strategies to handle incorrect and missed detections will be discussed further in Section 5.

OCCLUSION as an issue with the detector.

4.3 Similarity

To determine whether temporally correlated objects correspond to the same identity, we need to establish whether they are *sufficiently similar*, as described in Assumption 1. The degree of similarity between temporally correlated objects can be measured by comparing the objects IPPs using a similarity metric, as they are expected to experience minimal change. In this section we will discuss similarity metrics for position, appearance, and pose properties commonly used in TbD MOT systems.

Given a pair of temporally correlated detections, $d_1 = \{p_1, p_2, \dots, p_n\}$ and $d_2 = \{q_1, q_2, \dots, q_n\}$, with measured IPPs $p_i, q_i \in IPP_i$, we define a similarity metric $s_i(p_i, q_i)$ which determines the similarity between measured properties of an IPP IPP_i . Note that similarity metrics should be normalised i.e. constrained within the range between $[0, 1]$. This ensures consistency when combining different similarity metrics, which will be discussed further in Section 5.1.5.

4.3.1 Position

The **Intersection over Union (IoU)** metric is a widely used measure of similarity between two bounding boxes, b_1 and b_2 . It is calculated as the ratio of the area of overlap to the total area covered by both boxes:

$$s_{IoU}(b_1, b_2) = \frac{|b_1 \cap b_2|}{|b_1 \cup b_2|} \quad (1)$$

A higher IoU indicates greater overlap, making it an effective metric for determining object association [EPER20].

4.3.2 Appearance

To compare two feature vectors $\mathbf{a}, \mathbf{b} \in \mathbf{R}^n$, we can use the cosine distance defined as:

$$s_{\cos}(\mathbf{a}, \mathbf{b}) = 1 - \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|} \quad (2)$$

This metric measures the cosine of the angle between the two vectors, effectively comparing their direction rather than their magnitude. The cosine distance is particularly useful when re-matching lost tracks after long-term lost periods, when motion is less discriminative [WBP17].

4.3.3 Keypoints

To compute the similarity between poses we will be discussing the metric defined in *AlphaPose* [FLT⁺22].

The first step involves **normalizing the pose vectors**. This is done by resizing the bounding boxes of both poses to the same scale and determining the centre point of each bounding box. Each keypoint coordinate is then shifted by subtracting the centre point, resulting in the normalized pose vectors P_1 and P_2 , where:

$$P_i = \{\{x_{i_1}, y_{i_1}, c_{i_1}\}, \dots, \{x_{i_{17}}, y_{i_{17}}, c_{i_{17}}\}\}, \quad B_i \text{ is the enclosing normalized bounding box.}$$

The **pose distance function** $s_{\text{pose}}(P_i, P_j | \Lambda)$ in *AlphaPose* consists of two main components: **keypoint similarity** and **spatial distance**.

1. **Keypoint Similarity (KSim)**: This function performs a soft matching between keypoints, weighted by confidence scores. The keypoints are matched within bounding boxes scaled to $\frac{1}{10}$ of the original box. The hyperparameter σ_1 controls the influence of keypoint confidence on similarity:

$$\text{KSim}(P_i, P_j | \sigma_1) = \sum_{n=1}^{17} \left(\tanh\left(\frac{c_{in}}{\sigma_1}\right) \cdot \tanh\left(\frac{c_{jn}}{\sigma_1}\right) \cdot I(k_n \in B(k_n)) \right) \quad (3)$$

2. **Spatial Distance (HSim)**: The spatial distance between corresponding keypoints is computed using an exponential decay function, with σ_2 controlling the spatial scale:

$$\text{HSim}(P_i, P_j | \sigma_2) = \sum_{n=1}^{17} \exp\left(-\frac{(k_{in} - k_{jn})^2}{\sigma_2^2}\right) \quad (4)$$

Finally, these components are combined with a weighting parameter λ to produce the overall pose distance:

$$s_{\text{pose}}(P_i, P_j | \Lambda) = \text{KSim}(P_i, P_j | \sigma_1) + \lambda \cdot \text{HSim}(P_i, P_j | \sigma_2) \quad (5)$$

where $\Lambda = \{\sigma_1, \sigma_2, \lambda\}$ are the function parameters.

4.4 Dissimilarity

Based on Assumption 1, temporally correlated objects should exhibit high similarity scores if they share the same identity. This suggests that pairs of objects which have the highest similarity scores should correspond to the same identity i.e. **maximising similarity**. An equivalent, and more practical approach is to frame the problem in terms of **minimising dissimilarity**. The problem now involves *matching the object pairs that experienced the least change across timesteps*.

Given a pair of temporally correlated objects d_1 and d_2 , the dissimilarity is defined as the complement of their similarity. Where the similarity is based on a normalised similarity score $s(o_1, o_2)$.

$$\text{dissimilarity}(o_1, o_2) = 1 - s(o_1, o_2) \quad (6)$$

This definition ensures that when two objects are very similar ($s(o_1, o_2) \approx 1$), the dissimilarity is low ($\text{dissimilarity}(o_1, o_2) \approx 0$). Conversely, when objects are dissimilar ($\text{dissimilarity}(o_1, o_2) \approx 1$), then the objects are not similar ($s(o_1, o_2) \approx 0$).

Framing the problem in terms of dissimilarity is especially convenient when solving it as a Linear Assignment Problem (LAP), where minimisation is the natural objective [EPER20]. This will be further discussed in Section ??.

5 Tracking Component

The tracking component is responsible for matching the objects detected in the current timestep with the objects present in the previous timestep, in order to form a continuous reference of object identity. This matching process, known as **Association**, is a core task in object tracking. In this section, we will formalise the association process, define what constitutes a track, explain how tracks represent temporal sequences of detections corresponding to the same identity, and discuss how to manage the lifecycle of tracks. This includes track updates, maintenance, and eventual termination as objects disappear or are no longer visible.

5.1 Association

Association is the process of linking objects in the current timestep with objects in the previous timestep in an optimal way which preserves the identity of the objects. In MOT, association is typically modelled as a *temporal maximum matching problem using weighted bipartite graphs* [EPER20].

5.1.1 Formulating Association

A **bipartite graph** is a graph consisting of two disjoint sets of vertices, where edges only exist between vertices from different sets [Die05]. Formally, a bipartite graph $G = (V_1, V_2, E)$ consists of:

- Two sets of vertices, V_1 and V_2 , such that $V_1 \cap V_2 = \emptyset$.
- A set of edges $E = \{(v_1, v_2) \mid v_1 \in V_1, v_2 \in V_2\}$, connecting the vertices in V_1 with vertices in V_2 .

Association is inherently a temporal problem as it compares objects in consecutive timesteps, and bipartite graphs can be used to naturally model this relationship. At some timestep n , the objects detected in the current timestep

D_n and the objects present in the previous timestep D_{n-1} form two disjoint vertex sets. An edge $(d_{n-1}, d_n) \in E$ between a vertex $d_{n-1} \in D_{n-1}$ and $d_n \in D_n$, represents a potential match between the temporally correlated objects. Each edge is assigned a weight $w(d_{n-1}, d_n)$, which quantifies the likelihood that the two temporally correlated objects correspond to the same identity. The weight is derived from a similarity metric such as those discussed in Section ?? . The weight

A **complete** bipartite graph is one which every vertex from one set is connected to every vertex in the other set [Die05]. In the context of tracking, we define an **association graph** $A_n = (D_{n-1}, D_n, E)$ (Figure 1) as a complete weighted bipartite graph. This means that every object in the previous timestep D_{n-1} is connected by an edge to every objects in the current timestep D_n i.e. $E = D_{n-1} \times D_n$. The edge weights $w(d_{n-1}, d_n)$ are used to determine which object pairs are most likely to correspond to the same identity.

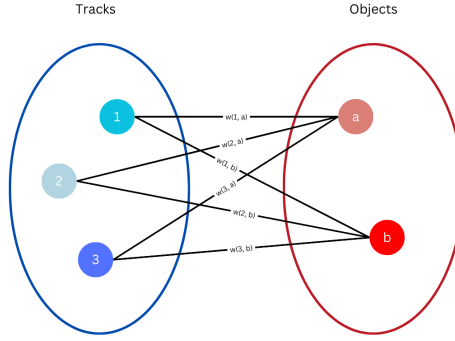


Figure 1: An example of a complete weighted bipartite graph, illustrating the connections between tracks and detections with corresponding edge weights.

A **matching** $M \subseteq E$ is a subset of edges where no two edges share a vertex. A **maximum matching** is one that contains the largest possible number of edges [CLRS09]. A **Maximum-Weight Matching** maximises the sum of the weights of its edges. Both maximum- and minimum-weight matchings are dual problems and can be used interchangeably to solve the same association problem [BC99].

The task of association can be framed as finding a minimum-weight matching in an association graph. Formally, we define association as Λ , which outputs a minimum-weight matching given an association graph A .

$$\Lambda(A_n) = \arg \min_{M \subseteq E} \sum_{(d_{n-1}, d_n) \in M} w(d_{n-1}, d_n). \quad (7)$$

Where $M \subseteq E$ is the set of edges in the matching, and $w(d_{n-1}, d_n)$ is the weight of the edge connecting the temporally correlated objects $d_{n-1} \in D_{n-1}$ and $d_n \in D_n$.

5.1.2 Solving Association

To solve the association problem, we reformulate the minimum-weight matching problem as an equivalent **Linear Assignment Problem** (LAP) [BC99]. The LAP involves assigning elements from one set to another set, such that each assignment incurs a certain cost. The objective is to find a bijection between two sets that minimises the total cost of assignment [BC99].

The association graph $A_n = (D_{n-1}, D_n, E)$ (defined in Section ??) can be represented as a cost matrix C_n , as the association graph is complete. The rows of the matrix correspond to the objects present at timestep $n - 1$ and the columns correspond to the objects detected in the current timestep n . The size of the matrix is given by $|C_n| = |D_{n-1}| \times |D_n|$. The elements of the cost matrix $c(d_{n-1}, d_n)$ represent the cost of assigning an object $d_{n-1} \in D_{n-1}$ present in the previous timestep to an object $d_n \in D_n$ detected in the current timestep. This cost corresponds to the weight of the edge in the association graph i.e. $c(d_{n-1}, d_n) = w(d_{n-1}, d_n)$.

Since we are dealing with a minimisation problem, the cost should be inversely related to the likelihood of a match i.e. a higher cost should indicate a lower likelihood of matching. As discussed in Section 4.4, dissimilarity also follows an inverse relationship with matching likelihood, and we can formulate the cost function based on the dissimilarity between temporally correlated objects, so dissimilar objects incur higher costs. Further details on constructing the cost function are provided in Section ?? .

With this formulation, the association function Λ can be expressed as a LAP. Where the function takes a cost matrix C_n (based on an association graph) as input and outputs a bijection $f : D_{n-1} \rightarrow D_n$ mapping temporally correlated objects, minimising the total assignment cost.

$$\Lambda(C_n) = \min_f \sum_{d_n \in D_n} c(f(d_{n-1}), d_n). \quad (8)$$

Efficient algorithms, such as the **Hungarian Algorithm**, can solve the LAP in polynomial time [Kuh55]. This makes the Hungarian algorithms well-suited for the task of online tracking and has been used in TbD MOT algorithms such as [WBP17] and [ZSJ⁺22] to solve the assignment problem.

5.1.3 limitations of Association

As described in Equation 8, the result of association Λ is a bijection that uniquely maps a detected object from the previous timestep to exactly one object detected in the current timestep. This inherently assumes that the number of objects to associate across timesteps is equal. This is rarely the case in real-world scenarios, as objects may enter or leave the scene (Section 3), or errors may occur with the detector (Section 4.2). In such cases, a bijection cannot be formed as there will be unmatched objects.

Given some association graph $A_n = (D_{n-1}, D_n, E)$ at some timestep n . In the case that there is an equal number of temporally-correlated objects to

associate i.e. $|D_{n-1}| = |D_n|$, the graph is referred to as **balanced**. This is the case in the classical LAP (Equation 8), where the cost matrix C_n is square, and a **perfect matching** is possible, ensuring that every object present in the previous timestep is uniquely matched with exactly one object detected in the current timestep [RT12].

In real-world tracking scenarios, the association graph is often **unbalanced**, where $|D_{n-1}| \neq |D_n|$. In this case the cost matrix C_n is rectangular, and cannot be solved using the classical LAP [RT12]. A naive approach to solve this issue is to augment the smaller set (either $|D_{n-1}| < |D_n|$ or $|D_n| < |D_{n-1}|$ could exist) with "dummy" objects that represent impossible matches i.e very high costs. This transforms the cost matrix C_n into a square matrix, allowing the classical LAP to be solved. The result is a **one-sided perfect** matching, where all objects of the smaller set are matched, while the objects in the larger set assigned to the dummy objects are considered unmatched. This method is employed in the Bytetrack algorithm [ZSJ⁺22].

The LAP, by design, matches every element in the sets, even when the matching costs between the objects are relatively high [BC99]. This may lead to forced matchings between temporally correlated objects that are dissimilar, which can result in **identity switches**. This is when temporally correlated objects corresponding to different identities are matched [LXM⁺21]. To mitigate this and maintain identity consistency as much as possible, we can make use of a **matching condition** that acts as a filtering mechanism to ensure that matches between temporally correlated objects only occur when the cost is below a certain threshold, such a technique was used in [ZSJ⁺22].

The threshold is a parameter that controls how much change is allowed across consecutive timesteps. A higher threshold allows for more flexibility in matching, potentially accepting more dissimilar matches, which may lead to more identity switches. While a lower threshold enforces stricter matching conditions and reduces the risk of incorrect associations. An adaptive threshold could also be explored, where the threshold adjusts dynamically based on factors such as the uncertainty of detections or the confidence in track consistency.

In association, there are two primary scenarios that can lead to objects being unmatched:

- performing association on an unbalanced association graph, where the number of tracks and detections are not equal.
- matches are filtered based on the matching condition, which rejects associations with costs exceeding a certain threshold.

These scenarios naturally lead to a partition of the temporally correlated object sets into matched and unmatched subsets.

5.1.4 Partitions

Given an association graph $A_n = (D_{n-1}, D_n, E)$, where D_{n-1} and D_n are disjoint sets of objects present in consecutive timesteps. Performing association

$\Lambda(C_n)$ on the cost matrix C_n yields a bijection $f_n : D_{n-1} \rightarrow D_n$, mapping matched objects in consecutive timesteps. All elements of D_{n-1} and D_n that are not mapped by f_n are considered unmatched. Thus, f_n induces a natural partition on the temporally correlated object into **matched** and **unmatched** subsets.

We denote the partition of a set D into its disjoint matched subset $M(D)$ and unmatched $U(D)$ subset as $P(D) = [M(D), U(D)]$, where $D = M(D) \cup U(D)$, such that $M(D) \cap U(D) = \emptyset$ [Die05].

The set of objects in the previous timestep D_{n-1} is partitioned into the matched subset $M(D_{n-1})$, which includes all objects in D_{n-1} mapped by f_n , and the unmatched subset $U(D_{n-1})$, which is the complement. This partition $P(D_{n-1})$ is defined as:

$$P(D_{n-1}) = [M(D_{n-1}), U(D_{n-1})]$$

where:

$$\begin{aligned} M(D_{n-1}) &= \{d_{n-1} \in D_{n-1} \mid f_n(d_{n-1}) \in D_n\}, \\ U(D_{n-1}) &= D_{n-1} \setminus M(D_{n-1}) \end{aligned}$$

Similarly, the set of objects in the current timestep D_n is partitioned into the matched subset $M(D_n)$, consisting of objects in D_n that are mapped by f_n^{-1} , and the unmatched subset $U(D_n)$, which is the complement. This partition $P(D_n)$ is defined as:

$$P(D_n) = [M(D_n), U(D_n)]$$

where:

$$\begin{aligned} M(D_n) &= \{d_n \in D_n \mid f_n^{-1}(d_n) \in D_{n-1}\}, \\ U(D_n) &= D_n \setminus M(D_n) \end{aligned}$$

Thus, the partition of the association graph $A_n = (D_{n-1}, D_n, E)$ is defined by the partitions of the object sets in consecutive timesteps, given the bijection $f_n = \Lambda(C_n)$:

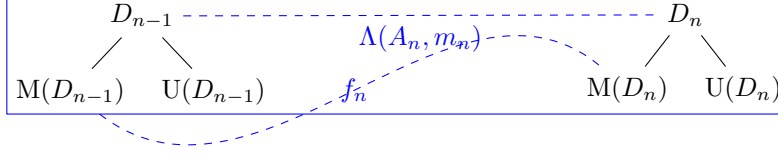
$$P(A_n, f_n) = [P(D_{n-1}), P(D_n)] = [[M(D_{n-1}), U(D_{n-1})], [M(D_n), U(D_n)]] \quad (9)$$

This leads to the overall partition of the association graph:

$$D_{n-1} \cup D_n = (M(D_{n-1}) \cup U(D_{n-1})) \cup (M(D_n) \cup U(D_n))$$

Since D_{n-1} and D_n are disjoint, and the matched and unmatched subsets are also disjoint, we have $M(D_{n-1}) \cap U(D_{n-1}) \cap M(D_n) \cap U(D_n) = \emptyset$.

We can equivalently view this as a tree diagram termed the **Association Tree**, which partitions associated the sets each into matched and unmatched elements. In the case of the general association defined in Equation 9, the association tree is as follows.



5.1.5 Constructing a Cost Function

In the association process, the cost function $c(d_{n-1}, d_n)$ quantifies the dissimilarity (or similarity) between temporally correlated objects d_{n-1} and d_n across consecutive timesteps. This function is essential for optimally matching objects, as described in Section 5.1.2. Since we are solving a minimization problem, the cost function should be inversely proportional to likelihood of matching. We can express the cost function in terms of dissimilarity between objects (Section ??), which is inversely related to the similarity metrics $s(d_{n-1}, d_n)$ defined in Section ??.

In this section, we outline different methods for constructing robust cost functions based on the properties of objects. Incorporating multiple properties helps improve matching accuracy by emphasizing different aspects of the objects and filtering out potential false matches. The key methods for constructing cost functions include **Weighted Sums**, **Gating**, and **Incorporating NIPPs**.

To formalize this, consider two objects from consecutive timesteps, $d_{n-1} = \{p_1, p_2, \dots, p_m\}$ and $d_n = \{q_1, q_2, \dots, q_m\}$, where p_i and q_i represent specific properties of the objects (such as position, appearance, or keypoints). We define a similarity function s that aggregates the individual similarity metrics $s_1, \dots, s_m$, where each $s_i : P_i \times P_i \rightarrow [0, 1]$ measures the similarity between corresponding properties. The cost function $c(d_{n-1}, d_n)$ is then derived by applying these similarity metrics, accounting for dissimilarity in the minimization process.

Weighted Sum In this method, a linear combination of normalised properties is computed using predefined weights. The weighted sum allows for fine-tuning the relative importance of each property, which is useful in cases where certain properties should have more influence on the similarity score than others.

A combined function can be constructed using a weighted average of the negated similarity scores (to get a value of dissimilarity).

$$c(d_{n-1}, d_n) = w_1(1 - s_1(p_1, q_1)) + w_2(1 - s_2(p_2, q_2)) + \dots + w_m(1 - s_m(p_m, q_m)) \quad (10)$$

where w_i are normalised weights, $\sum_{i=1}^m w_i = 1$. These weights can be adjusted to reflect the relative importance of each property.

Gating Gating is a technique where the similarity score of one property is threshold based on the value of another. This helps in reducing False Positives by enforcing stricter matching criteria. For example, in SMILETrack [WHC⁺24], gating is used to reject object pairs if their appearance similarity is below a threshold, even when their positions are relatively close. This approach is particularly useful when certain properties, such as appearance, are more indicative of identity than others, like position.

Consider two temporally correlated objects $d_{n-1} = \{p_1, p_2\}$ and $d_n = \{q_1, q_2\}$ with properties $p_1, q_1 \in P_1$ and $p_2, q_2 \in P_2$. If $s_1(p_1, q_1) = 0.9$ and $s_2(p_1, q_2) = 0.1$, the objects are similar w.r.t P_1 but differ significantly w.r.t P_2 . Gating ensures objects must be similar across all properties to be considered a match.

Formally, the gating mechanism can be defined as:

$$g(d_{n-1}, d_n) = \begin{cases} 0 & \text{if } s_i(p_i, q_i) > g_{thres}^i \text{ for some } i \\ 1 & \text{otherwise} \end{cases} \quad (11)$$

where g_{thres}^i is the threshold for property P_i . We can now define a combined similarity function which incorporates a gating mechanism, where $c(d_{n-1}, d_n)$ is a general cost function.

$$c_g(d_{n-1}, d_n) = \begin{cases} \infty & \text{if } g(d_{n-1}, d_n) = 0 \\ c(d_{n-1}, d_n) & \text{otherwise} \end{cases} \quad (12)$$

By using this gated cost function, we ensure that object pairs that are not similar with respect to some property are assigned an infinite cost, effectively not considering this pair as a viable match.

Incorporating NIPPs NIPPs, such as confidence scores, can be incorporated into the similarity metric to further refine the matching process. For example, in SMILETrack [WHC⁺24], the confidence score of a detection is multiplied with the position similarity (e.g., IoU score) between two objects. This results in a weighted similarity score where high-confidence detections are given preference during association. Incorporating NIPP can thus improve the accuracy and reliability of the tracking system by weighting scores according to the reliability of the detections.

5.2 Track

Association compares objects present in the previous timestep D_{n-1} with objects detected in the current timestep D_n , based on an association graph $A_n = (D_{n-1}, D_n, E)$, as discussed in Section 5.1.1. In the tracking process, we use association to propagate identity across timesteps. To be able to propagate identity, the objects from previous timesteps need to maintain some notion of identity.

5.2.1 Formalising Track and Trajectory

The unit of identity is referred to as a **track**. A track t^i corresponds to a unique object identity denoted by a natural number $i \in \mathbf{N}$. The trajectory $\tau^i : \mathbf{N} \rightarrow \mathbf{D}$ of a track t^i maps timesteps $\mathbf{N}$ to the set detected objects $\mathbf{D}$, serving as a reference for detected objects corresponding to the same object identity at different timesteps. Formally, a track t^i can be defined as $t^i = (i, \tau^i, s^i)$, with id i , trajectory τ^i , and state s^i which will be discussed in more detail in Section 5.2.3.

To incorporate identity into the association process, we compare tracks instead of the raw detections D_{n-1} . Each detection $d_{n-1} \in D_{n-1}$ should correspond to a unique object identity, and so to some track t^i . Furthermore, each detected object $d_{n-1} \in D_{n-1}$ should be the *most recent* reference of object identity within its corresponding trajectory τ^i .

$$d_{n-1} = \tau^i(m), \quad m \leq n-1 \quad (13)$$

It is important to note that the most recent element in a trajectory does not always correspond to the detection from the immediately preceding timestep $n-1$. In some cases, the trajectory may not be updated for certain timesteps. However, as long as the most recent detection in the trajectory in the previous timestep $m \leq n-1$ is **temporally correlated** with the current detections in timestep n , then the most recent detection can be considered for association.

To maintain reliable temporal correlation, a threshold is introduced, which defines how far back in time we can consider the most recent detection of some identity for association. If the difference in time $n-m$ exceeds this threshold, the most recent detection in the track at timestep m is said to not be temporally correlated with the current timestep n , and the track can no longer be associated with future detections. This threshold is a system parameter and controls the trade-off between temporal consistency and the likelihood of identity switches. A higher threshold allows for less temporally correlated associations, which could increase identity switches, while a lower threshold results in more temporally correlated trajectories but excludes more tracks from future associations.

5.2.2 Tracking as Temporal Association of Identity

We can perform association between the most recently detected object in the trajectories of tracks T_n at timestep n , which reference the detections D_{n-1} present at the previous timestep, and the current detections D_n , provided they are temporally correlated (i.e $n-m \leq tc$, where tc is the temporal correlation threshold). Given this, we can represent the relationships in the association graph $A_n = (D_{n-1}, D_n, E)$ based on the trajectories of objects. Given a detection present in the previous timestep $d_{n-1} \in D_{n-1}$, then by 13, $d_{n-1} = \tau^i(m)$, for some track t^i , with identity i , at timestep m .

This implies that association graph can be represented in terms of trajectories, such that $(\tau^i(m), d_n) \in E$, with weights $w(\tau^i(m), d_n)$. This also implies

that the elements of the cost matrix C_n is the cost of assigning the most recent detection of some identity with the new detections since $w(\tau^i(m), d_n) = w(\tau^i(m), d_n)$.

Therefore, the tracks T_n reference objects detections present at the previous timestep D_{n-1} through their trajectories. Thus the association graph A_n , can be used to perform association between tracks T_n and detections D_n , by referencing the latest detection in each track's trajectory. This process partitions the sets D_{n-1} and D_n , as explained in Section ?? . Since tracks T_n reference detections in D_{n-1} , they are similarly partitioned into **matched** and **unmatched** tracks.

Matched tracks are updated with the corresponding new detection, while unmatched tracks remain unchanged.

$$\tau^i(n) = \begin{cases} f_n(\tau^i(m)) & \text{if } \tau^i(m) \in M_{n-1} \\ \emptyset & \text{if } \tau^i(m) \in U_{n-1} \end{cases} \quad (14)$$

This update rule highlights how tracks evolve over time. If a track's last detection was matched, its trajectory is updated with the new detection. Otherwise, if unmatched, the trajectory remains empty for that timestep.

This implies that the bijection obtained during association maps consecutive elements in the trajectory. Given the association graph $A_n = (D_{n-1}, D_n, E)$ where $\tau(i)(m) = d_m \in D_{n-1}$ and $\tau(i)(n) = d_n \in D_n$, the bijection $f_n = \Lambda(C_n)$, uniquely maps the consecutive elements of the trajectory, $f_n(d_m) = d_n$. Based on this, the recurrent relationship in the trajectory can be established:

$$f_n(\tau(i)(m)) = \tau(i)(n) \quad (15)$$

where $m < n$, but $\tau(i)(m)$ is the latest element in the trajectory.

5.2.3 Track State

We formulate tracking as a temporal identity association problem, where at each timestep n , tracks T_n are associated with new detections D_n . Given a track $t^i = (i, \tau^i, s^i) \in T_n$, its trajectory evolves based on the association procedure, as described in Equation 14.

If a track is successfully matched, the trajectory is updated accordingly. However, when a track fails to match, several factors may be responsible, including occlusion, missed detections, or identity switches due to incorrect assignments. The system must also manage false positives that do not correspond to real objects and tracks that are no longer temporally correlated.

To address these challenges, we model each track t^i as a state machine with four possible states: *NEW*, *MATCHED*, *LOST*, and *RESERVED*.

Reserved The *RESERVED* state is both the initial and final stage of the tracking process. A track in this state is inactive and does not participate in tracking, either temporarily or permanently. Upon initialization, a track enters the *RESERVED* state as it has no assigned identity. It returns to this state

when the tracking system determines that its identity can no longer be reliably maintained across timesteps.

New A *NEW* track represents an unverified detection that has yet to be confirmed as a consistent object identity. This state serves as a provisional phase to validate detections and mitigate false positives.

Matched The *MATCHED* state represents reliable object tracking. A track in this state has consistently been associated with detections across timesteps, maintaining identity continuity. Tracks in this state are considered the most reliable for future associations.

Lost The *LOST* state handles identity disruptions caused by occlusions or missed detections. A track remains *LOST* for a limited duration, allowing for potential re-association. However, as time progresses, its temporal consistency weakens, reducing the likelihood of a match. If its consistency becomes unreliable, the track is not considered during the tracking process.

The state of a track, denoted as s^i , evolves over time and is represented as a mapping:

$$s^i : \mathbf{N} \rightarrow \{NEW, MATCHED, LOST, RESERVED\} \quad (16)$$

This mapping describes how the track’s state updates as its trajectory progresses across timesteps $n \in \mathbf{N}$. The state transition at each timestep n depends on the outcome of the association process. If the track is successfully associated with a detection ($\tau^i(n) \neq \emptyset$), it performs a **match transition** (M). Otherwise, if no detection is assigned ($\tau^i(n) = \emptyset$), it performs an **unmatch transition** (U), as defined in Equation 14.

5.2.4 Track Lifecycle

Building on the concept of track states introduced in Section 5.2.3, the lifecycle of a track can be modelled as a finite-state automaton. This automaton defines the evolution of a track’s state across time, driven by the association of detections and the corresponding transitions. The track lifecycle is depicted in Figure 2.

The lifecycle of a track is driven by specific **lifecycle functions** which are conditional, case-dependent rules that determine when and how a track moves between the states. The lifecycle functions include:

- **Creation:** The track is created when it is first associated with a detection, marking the beginning of its identity.
- **Activation:** A track transitions from *NEW* to *MATCHED* when it is confirmed to be a consistent object identity, likely to be matched again.

- **Deactivation:** If a track fails to reappear after being temporarily lost (e.g., due to occlusion or missed detections), it enters the *RESERVED* state, ending its active tracking.
- **Reactivation:** A track in the *RESERVED* state can be reactivated if it reappears and is successfully matched with a detection, moving back to the *MATCHED* state.

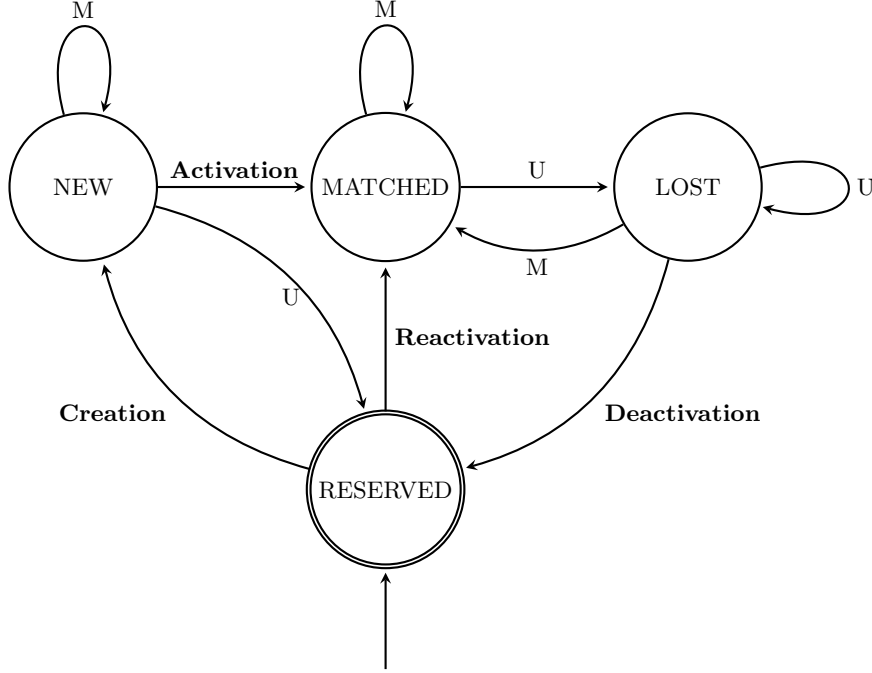


Figure 2: Track Lifecycle

A track begins its lifecycle in the *RESERVED* state, indicating that it is not yet associated with an object identity. It is considered **created** when an initial detection is linked to the track, forming the basis for its identity. The conditions for creation depend on the tracking system. For example, ByteTrack [ZSJ⁺22] creates tracks only if the detection confidence exceeds a predefined threshold. Incorporating NIPPs into lifecycle functions can further refine tracking results.

Upon creation, the track moves to the *NEW* state, where it is subject to validation. If it consistently matches detections over successive timesteps, the track is **activated** and transitions to the *MATCHED* state. However, if the track remains unmatched before activation, it transitions back to the *RESERVED* state and not considered a reliable object identity.

In the *MATCHED* state, a track is considered reliable, maintaining identity continuity. However, if it fails to match new detections—due to occlusion,

missed detections, or tracking errors—it transitions to the *LOST* state. A track remains in the *LOST* state for a limited period, allowing for potential recovery. This period is determined by a temporal consistency threshold, which defines the maximum allowable gap between detections. Specifically, based on Equation 15, a track is **deactivated** if the time elapsed since its last successful match, given by $n - m$, exceeds this threshold. This user-defined parameter controls how long a track remains eligible for re-association before being removed from active tracking.

If a *LOST* track cannot be recovered, it transitions back to the *RESERVED* state, effectively deactivating it. Depending on the system, a *RESERVED* track may be permanently removed or subject to **reactivation**. Systems utilizing re-identification (Re-ID), such as StrongSORT [DZS⁺23], can reactivate previously matched identities, allowing for tracking continuity even when an object temporarily leaves the scene. This is particularly useful in scenarios where individuals exit and later return. In both cases, the deactivation condition ensures that tracks unlikely to contribute to future detections—whether due to missed matches or false positives—are discarded or placed on hold.

5.2.5 Prediction

A Kalman Filter is a recursive mathematical technique that integrates predictions from a dynamic model with noisy measurements to estimate the state of objects in motion [LRJD15]. In the context of object tracking, Kalman Filters are employed to predict the positions of tracks from previous frames, facilitating a more accurate comparison with detected objects in the current frame. This predictive capability improves the robustness of object association across video frames by aligning expected object trajectories with real-time observations. Notably, tracking algorithms like *DeepSORT* [WBP17] and *BYTEtrack* [ZSJ⁺22] integrate Kalman Filters to enhance tracking accuracy and reliability.

5.3 Tracking

5.3.1 Cascaded Assignment

Priority-Based Cascading Association (PBCA) is a generalisation of techniques used in various online tracking-by-detection systems to improve the accuracy and robustness of object tracking. This method has been applied under different names in multiple works, including *Matching Cascade* [WBP17], *Multi-Stage Identity Matching* [FLT⁺22], *BYTE* [ZSJ⁺22], and *Similarity Matching Cascade* [WHC⁺24].

The principle idea of PBCA is to partition one of the sets in association (either tracks or detections) based on some NIPP, such as confidence scores [ZSJ⁺22], or the age of the track (the number of timesteps a track has consistently matched) [WBP17]. This induces a natural ordering of association on the partitions. Repeated association is then performed between the partitions of the Assignment Set (based on Equation 9) and each partition of the Partitioned Set

starting from the highest priority to the lowest priority, constantly refining the list of unmatched detections.

Higher priority partitions should consist of objects which have a higher likelihood of matching during association, based on some heuristic. In *BYTE-track* [ZSJ⁺22], detections are split into two partitions based on their confidence scores. High-confidence detections are matched with tracks first, followed by lower-confidence detections. *DeepSORT* [WBP17] partitions tracks based on their age, prioritizing the association of tracks that have been consistently matched over time. Prioritisation is useful as it focuses on the most likely matches first, thereby reducing the chances of incorrect associations.

Algorithm 1: Cascaded Assignment

Input: Tracks T ,
Detection Partitions $\{D_1, D_2, \dots, D_k\}$, $D_i \cap D_j = \emptyset$,
Matching thresholds $\{m_1, m_2, \dots, m_k\}$
Output: Matched tracks T_M , Unmatched tracks T_U ,
Matched detections D_M , Unmatched detections D_U

```

1  $T_M \leftarrow \emptyset$ ;
2  $D_M \leftarrow \emptyset$ ;
3  $T_U \leftarrow T$ ;
4  $D_U \leftarrow \emptyset$ ;
5 for  $i \leftarrow 1$  to  $k$  do
6    $(T_{M_i}, T_{U_i}, D_{M_i}, D_{U_i}) \leftarrow \text{Assignment}(T_U, D_i, m_i)$ ;
7    $T_M \leftarrow T_M \cup T_{M_i}$ ;
8    $D_M \leftarrow D_M \cup D_{M_i}$ ;
9    $T_U \leftarrow T_{U_i}$ ;
10   $D_U \leftarrow D_U \cup D_{U_i}$ ;
11 end for
12 return  $T_M, T_U, D_M, D_U$ 

```

At each iteration of the cascade, the unmatched objects of the assignment set in the previous iteration are associated with the corresponding partition, further refining the assignment set. The output of the association cascade is the fully refined unmatched assignment set A_U , the matched objects at each iteration between the assignment set and the partition set A_M and P_M . Finally, the unmatched objects of the partition set at each partition P_U .

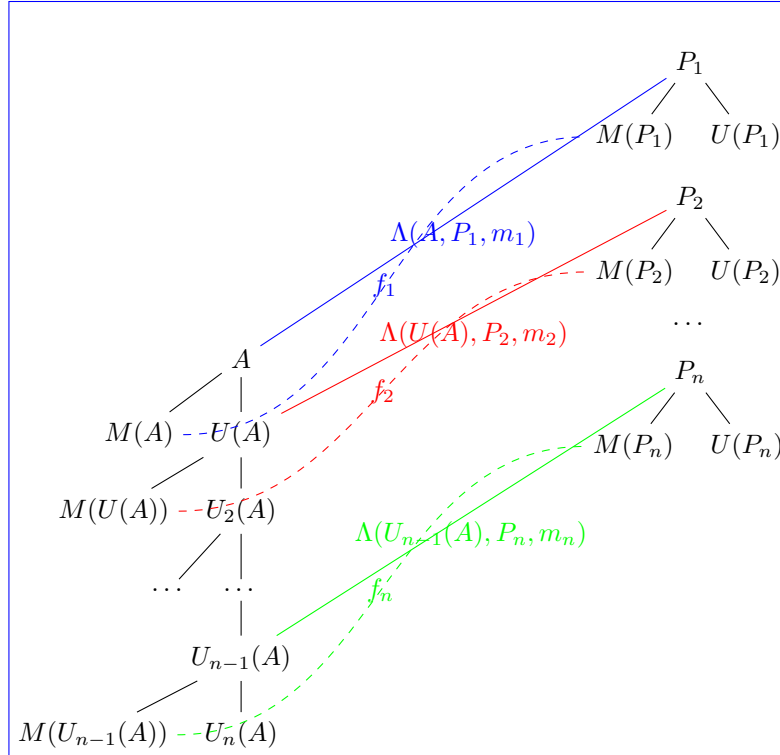
Algorithm 2: Cascaded Assignment Algorithm

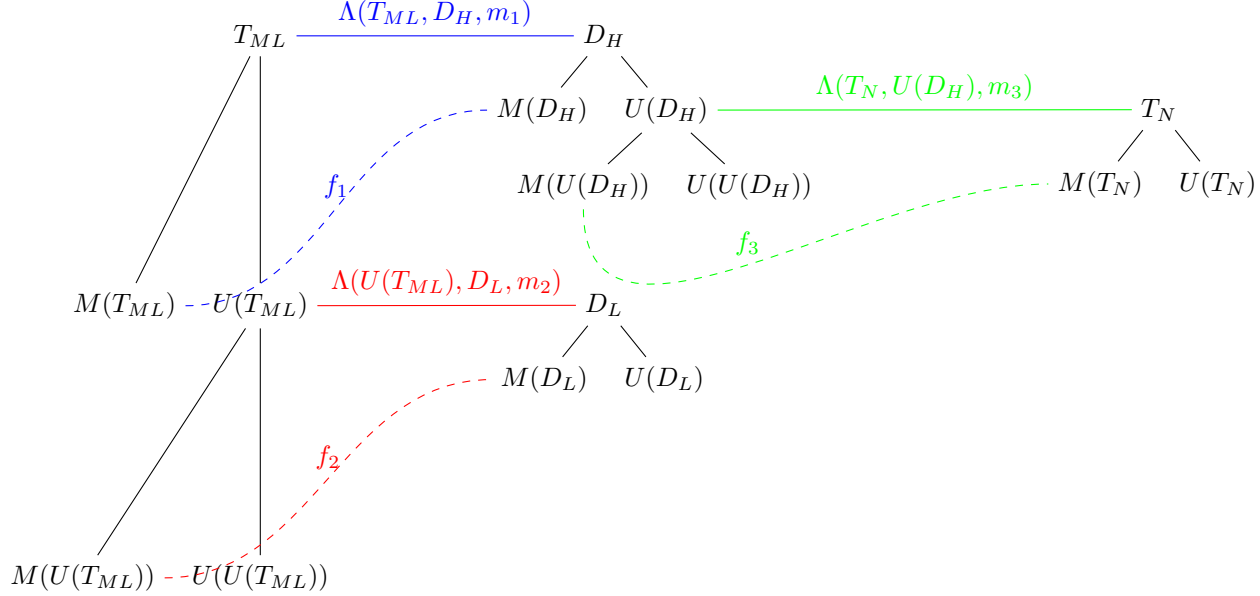
Input: Assignment Set A , Partitioned Set $P = \bigcup_{i=1}^k P_i$, where $\bigcap_{i=1}^k P_i = \emptyset$, Matching thresholds list $\{m_1, m_2, \dots, m_k\}$
Output: Matched assignment set A_M , Unmatched assignment set A_U , Matched partition set P_M , Unmatched partitioned set P_U

```

1 Function CascadedAssignment( $A, P, \{m_1, m_2, \dots, m_k\}$ ):
2    $A_M, P_M \leftarrow \emptyset$  // Initialise matched sets
3    $A_U \leftarrow A$  // Initialise unmatched assignments
4    $P_U \leftarrow \emptyset$  // Initialise unmatched partitions
5   for  $i \leftarrow 1$  to  $k$  do
6     /* Perform association between remaining unmatched
       assignments and partition  $P_i$  */
7      $[[MU_{i-1}(A), U_i(A)], [M(P_i), U(P_i)]] \leftarrow \Lambda(A_U, P_i, m_i)$ 
8      $A_M \leftarrow A_M \cup MU_{i-1}(A)$  // Update matched assignments
9      $A_U \leftarrow U_i(A)$  // Update unmatched assignments
10     $P_M \leftarrow P_M \cup M(P_i)$  // Update matched partitions
11     $P_U \leftarrow P_U \cup U(P_i)$  // Update unmatched partitions
12  end for
13  return  $A_M, A_U, P_M, P_U$ 

```





5.3.2 General Tracking Procedure

The tracker functions as a state manager, responsible for updating track states through association. By leveraging **PBCA** (Section 5.3.1), we can apply different association methods designed for various stages of the tracking process.

This section presents a general tracking algorithm that adapts its association strategy based on both the track state and the number of tracks in each state at a given time step. The tracker associates detections from the detection component with internally stored tracks, updating their states accordingly. Tracks are then reassigned to their respective state groups—if a previously matched track becomes lost, it is moved to the **LOST** state. Finally, **LOST** tracks that meet the deactivation criteria are removed from the system to maintain efficiency.

At each timestep, tracks in the **NEW**, **MATCHED**, and **LOST** states are eligible for association with new detections. However, tracks in the **RESERVED** state may not always participate in the matching process, depending on whether **re-association** is implemented.

If no tracks exist in the **NEW**, **MATCHED**, or **LOST** states, the tracker initiates a track creation step, generating new tracks based on a predefined creation condition.

Initial Tracking: When only **NEW** tracks are present, a simple association is performed between them and the current detections. Successful matches promote tracks to the **MATCHED** state, while unmatched tracks may be placed in the **RESERVED** state.

Subsequent Tracking: When **MATCHED** or **LOST** tracks exist, a more refined, multi-stage association process is applied. Detections are often parti-

tioned (e.g., by confidence score), and a cascaded association strategy is used to match tracks in a structured manner.

The choice of association algorithms at each stage is guided by the reliability and characteristics of different track states:

- **Step 1: Prioritizing MATCHED and LOST tracks** – These tracks have historical trajectory data, making them more stable and well-suited for robust matching algorithms such as the Hungarian algorithm or graph-based methods.
- **Step 2: Reassociating unmatched tracks** – Tracks that remain unmatched in the first stage undergo a second round of association, using a lower matching threshold to recover **LOST** tracks that may have disappeared due to occlusions or detection failures.
- **Step 3: Assigning new detections** – Any remaining unmatched detections are assigned to **NEW** tracks, ensuring that all observed objects are incorporated into the tracking system.

This structured approach enhances tracking accuracy by prioritizing stable matches while allowing for track recovery and initialization, leading to a more robust and reliable tracking system.

5.4 Conclusion

References

- [AOB22] Nir Aharon, Roy Orfaig, and Ben-Zion Bobrovsky. Bot-sort: Robust associations multi-pedestrian tracking, 2022.
- [BC99] Rainer E Burkard and Eranda Cela. Linear assignment problems and extensions. In *Handbook of combinatorial optimization: Supplement volume A*, pages 75–149. Springer, 1999.
- [BGO⁺16] Alex Bewley, Zongyuan Ge, Lionel Ott, Fabio Ramos, and Ben Upcroft. Simple online and realtime tracking. In *2016 IEEE International Conference on Image Processing (ICIP)*. IEEE, September 2016.
- [CLRS09] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms, fourth Edition*. The MIT Press, 4th edition, 2009.
- [CPW⁺23] Jinkun Cao, Jiangmiao Pang, Xinshuo Weng, Rawal Khirodkar, and Kris Kitani. Observation-centric sort: Rethinking sort for robust multi-object tracking, 2023.
- [DHZL22] Yan Dai, Ziyu Hu, Shuqi Zhang, and Lianjun Liu. A survey of detection-based video multi-object tracking. *Displays*, 75:102317, 10 2022.

- [Die05] Reinhard Diestel. *Graph Theory (Graduate Texts in Mathematics)*. Springer, August 2005.
- [DOM⁺20] Patrick Dendorfer, Aljoša Ošep, Anton Milan, Konrad Schindler, Daniel Cremers, Ian Reid, Stefan Roth, and Laura Leal-Taixé. Motchallenge: A benchmark for single-camera multiple target tracking, 2020.
- [DRM⁺20] Patrick Dendorfer, Hamid Rezatofighi, Anton Milan, Javen Shi, Daniel Cremers, Ian Reid, Stefan Roth, Konrad Schindler, and Laura Leal-Taixé. Mot20: A benchmark for multi object tracking in crowded scenes, 2020.
- [DZS⁺23] Yunhao Du, Zhicheng Zhao, Yang Song, Yanyun Zhao, Fei Su, Tao Gong, and Hongying Meng. Strongsort: Make deepsort great again, 2023.
- [EPER20] Patrick Emami, Panos M. Pardalos, Lily Elefteriadou, and Sanjay Ranka. Machine learning methods for data association in multi-object tracking. *ACM Computing Surveys*, 53(4):1–34, August 2020.
- [FLT⁺22] Hao-Shu Fang, Jiefeng Li, Hongyang Tang, Chao Xu, Haoyi Zhu, Yuliang Xiu, Yong-Lu Li, and Cewu Lu. Alphapose: Whole-body regional multi-person pose estimation and tracking in real-time, 2022.
- [Kuh55] Harold W. Kuhn. The hungarian method for the assignment problem. *Naval Research Logistics (NRL)*, 52, 1955.
- [LRJD15] Qiang Li, Li Ranyang, Kaifan Ji, and Wei Dai. Kalman filter and its application. pages 74–77, 11 2015.
- [LXG22] Yiyi Liao, Jun Xie, and Andreas Geiger. Kitti-360: A novel dataset and benchmarks for urban scene understanding in 2d and 3d, 2022.
- [LXM⁺21] Wenhan Luo, Junliang Xing, Anton Milan, Xiaoqin Zhang, Wei Liu, and Tae-Kyun Kim. Multiple object tracking: A literature review. *Artificial Intelligence*, 293:103448, April 2021.
- [NH19] Guanghan Ning and Heng Huang. Lighttrack: A generic framework for online top-down human pose tracking, 2019.
- [RT12] Lyle Ramshaw and Robert E Tarjan. A weight-scaling algorithm for min-cost imperfect matchings in bipartite graphs. In *2012 IEEE 53rd Annual Symposium on Foundations of Computer Science*. IEEE, October 2012.
- [SCJ⁺22] Peize Sun, Jinkun Cao, Yi Jiang, Zehuan Yuan, Song Bai, Kris Kitani, and Ping Luo. Dancetrack: Multi-object tracking in uniform appearance and diverse motion, 2022.

- [WBP17] Nicolai Wojke, Alex Bewley, and Dietrich Paulus. Simple online and realtime tracking with a deep association metric, 2017.
- [WHC⁺24] Yu-Hsiang Wang, Jun-Wei Hsieh, Ping-Yang Chen, Ming-Ching Chang, Hung Hin So, and Xin Li. Smiletrack: Similarity learning for occlusion-aware multiple object tracking, 2024.
- [ZML⁺23] Lijuan Zhou, Xiang Meng, Zhihuan Liu, Mengqi Wu, Zhimin Gao, and Pichao Wang. Human pose-based estimation, tracking and action recognition with deep learning: A survey, 2023.
- [ZSJ⁺22] Yifu Zhang, Peize Sun, Yi Jiang, Dongdong Yu, Fucheng Weng, Zehuan Yuan, Ping Luo, Wenyu Liu, and Xinggang Wang. Bytetrack: Multi-object tracking by associating every detection box, 2022.